

RISING WATERS: AI-Based Flood Prediction System Using Machine Learning

Project Description:

Rising Waters is an Artificial Intelligence and Machine Learning-based flood prediction system designed to provide early flood risk assessment using historical environmental data. The project aims to assist disaster management authorities, local communities, and government agencies by predicting the likelihood of floods based on critical environmental parameters such as rainfall, water level, humidity, and temperature.

The project is developed using Python, Jupyter Notebook, and popular Machine Learning libraries including Pandas, NumPy, Scikit-learn, Matplotlib, and Seaborn. The completed project is maintained through GitHub for version control and demonstration purposes, making it easy to deploy, update, and extend in the future.

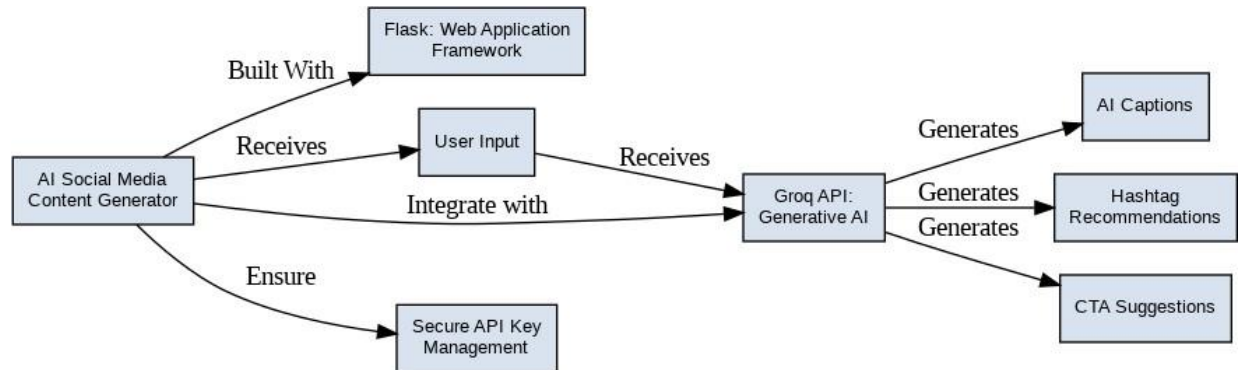
Scenarios

Scenario 1: A meteorological department records continuous heavy rainfall over several days in a particular region. Officials enter the rainfall amount, humidity, temperature, and water level into the Rising Waters system. The trained KNN model analyzes the environmental conditions and predicts a high probability of flooding. Based on this prediction, authorities issue early warnings and begin evacuation planning to reduce the impact of potential floods.

Scenario 2: During the monsoon season, river water levels begin rising rapidly. Disaster management personnel monitor the environmental conditions and use the Rising Waters application to predict flood risk. The model classifies the situation as **Flood Likely**, enabling local authorities to prepare emergency shelters, coordinate rescue teams, and alert nearby communities before the flood occurs.

Scenario 3: A resident living near a flood-prone river wants to understand the flood risk after several days of intense rainfall. By entering rainfall, water level, humidity, and temperature values into the application, the system predicts the likelihood of flooding. The user receives immediate results and can take necessary precautions such as safeguarding belongings, relocating livestock, or temporarily evacuating the area.

Technical Architecture:



Description: The Rising Waters system follows a modular machine learning architecture consisting of four major components: **Data Collection**, **Data Processing**, **Machine Learning Model**, and **Prediction & Visualization**. The **frontend** (or notebook interface) accepts environmental inputs such as rainfall, humidity, temperature, and water level. The **backend**, developed in Python, preprocesses the input data and sends it to the trained K-Nearest Neighbors (KNN) model for prediction. The prediction results are then displayed to the user along with graphical visualizations to improve understanding of flood risk.

Pre-requisites:

- Python Programming Knowledge
- Machine Learning Fundamentals
- Pandas Library for Data Processing
- NumPy for Numerical Computation
- Scikit-learn for Machine Learning Models
- Matplotlib & Seaborn for Data Visualization
- Jupyter Notebook or VS Code
- Git and GitHub for Version Control
- Basic Knowledge of Environmental and Flood Data
- Operating System: Windows/Linux
- Python Version: 3.10 or above

Project Workflow:

Milestone 1: Dataset Collection and System Architecture

- **Activity 1.1:** Collect a reliable historical flood dataset containing environmental parameters such as rainfall, humidity, temperature, and water level.
- **Activity 1.2:** Study various Machine Learning algorithms suitable for flood prediction and select the most appropriate model based on accuracy and simplicity.
- **Activity 1.3:** Design the overall architecture of the application, defining the interaction between the dataset, preprocessing module, machine learning model, and prediction interface.
- **Activity 1.4:** Set up the development environment by installing Python, Jupyter Notebook, required libraries, and configuring the project folder structure.

Milestone 2: Data Processing and Model Development

- **Activity 2.1:** Perform data preprocessing, exploratory data analysis, feature selection, and visualization of the flood dataset.
- **Activity 2.2:** Develop and train the K-Nearest Neighbors (KNN) Machine Learning model using the processed dataset and evaluate its performance.

Milestone 3: Model Implementation

- **Activity 3.1:** Implement the prediction logic, allowing users to input environmental parameters and obtain flood prediction results using the trained KNN model.

Milestone 4: User Interface Development

- **Activity 4.1:** Design a simple and user-friendly interface for entering environmental parameters and viewing prediction results.
- **Activity 4.2:** Display graphical visualizations such as scatter plots, heatmaps, and prediction outputs for better understanding of flood risk.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the machine learning model using unseen test data and verify the prediction accuracy.
- **Activity 5.2:** Upload the project to GitHub, document the source code, and prepare a demonstration video showcasing the working of the application.

Milestone 6: Conclusion

Summarize the project outcomes, evaluate the effectiveness of the Machine Learning model, discuss future enhancements, and document the overall contribution of the Rising Waters system toward flood prediction and disaster preparedness.

Milestone 1: Dataset Collection and System Architecture

The first milestone focuses on collecting a suitable historical flood dataset and establishing the overall architecture of the Rising Waters project. During this stage, environmental parameters such as rainfall, humidity, temperature, and water level are identified as the key features influencing flood occurrence. Publicly available datasets are collected and examined for completeness and relevance.

Activity 1.1: Collect Historical Flood Dataset

- The first activity involves gathering historical environmental data required for training the Machine Learning model. The dataset contains important parameters that influence flood occurrence and serves as the primary source of information for model development.

Steps Followed

- **Step 1 – Dataset Collection**

Obtain a publicly available flood prediction dataset from reliable sources such as Kaggle or government open-data portals.

- **Step 2 – Dataset Inspection**

Open the dataset using Pandas and examine its structure, number of records, column names, and data types.

```
import pandas as pd

df = pd.read_csv("flood_prediction_dataset.csv")

print(df.head())

print(df.info())

print(df.describe())
```

- **Step 3 – Verify Data Quality**

- Check for missing values.
- Remove duplicate records.
- Ensure numerical values are stored in the correct format.
- Confirm that the target variable (Flood) is correctly labeled.

- **Step 4 – Save the Clean Dataset**

After verification, save the cleaned dataset for preprocessing and model training.

Activity 1.2: Research and Select the Machine Learning Algorithm

Several Machine Learning algorithms were studied before selecting the final model for flood prediction. The algorithms considered include:

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)

After comparing their characteristics, the K-Nearest Neighbors (KNN) algorithm was selected because it is simple to implement, performs well for classification tasks, and provides satisfactory prediction accuracy for the available dataset.

Reasons for Selecting KNN

- Easy to understand and implement.
- Effective for binary classification problems.
- No complex model training process.
- Suitable for medium-sized datasets.
- Produces reliable prediction results after proper preprocessing.

Activity 1.3: Define the Architecture of the Application

The Rising Waters system follows a modular architecture consisting of five major components.

Modules

- Dataset Collection Module
- Data Preprocessing Module
- Machine Learning Module
- Prediction Module
- Visualization Module

Activity 1.4: Set Up the Development Environment

The development environment is configured before implementing the Machine Learning model.

Software Requirements

- **Python 3.10+**
- **Jupyter Notebook**
- **Visual Studio Code**
- **Git**
- **GitHub**

```
pip install pandas
pip install numpy
pip install matplotlib
pip install seaborn
pip install scikit-learn
pip install jupyter
```

- **Install**

```
pip install -r requirements.txt
```

Milestone 2: Data Processing and Model Development

This milestone focuses on preparing the collected dataset for machine learning and developing the flood prediction model. Data preprocessing is performed to improve data quality by handling missing values, removing duplicate records, selecting important features, and transforming the data into a suitable format for model training. Exploratory Data Analysis (EDA) is carried out to understand the relationships among environmental variables such as rainfall, humidity, temperature, and water level.

Activity 2.1: Perform Data Preprocessing and Exploratory Data Analysis

Proper preprocessing is essential for improving the prediction capability of the machine learning model. The following steps are performed before model training.

Load the Dataset

```
import pandas as pd  
  
df = pd.read_csv("flood_prediction_dataset.csv")  
  
df.head()
```

Check Dataset Information

```
df.info()  
df.describe()
```

Activity 2.2: Develop the Machine Learning Model

After preprocessing, the K-Nearest Neighbors algorithm is used to train the flood prediction model.

Milestone 3: App.py Development

Milestone 3 focuses on implementing the core logic of the Rising Waters Flood Prediction System. In this stage, the machine learning model developed during the previous milestone is integrated into the application workflow. The system accepts environmental parameters such as rainfall, water level, humidity, and temperature, processes the input using the trained K-Nearest Neighbors (KNN) model, and generates a prediction indicating whether flood conditions are likely to occur.

Activity 3.1: Writing the Main Application Logic in app.py

The main application logic of the Rising Waters Flood Prediction System is responsible for loading the flood dataset, preprocessing the data, training the Machine Learning model, evaluating its performance, and predicting flood occurrence based on environmental parameters.

```
def predict():
    rainfall = float(request.form["rainfall"])
    water = float(request.form["water"])
    humidity = float(request.form["humidity"])
    temp = float(request.form["temp"])

    sample = np.array([[rainfall, water, humidity, temp]])
    prediction = model.predict(sample)

    result = "Flood Likely" if prediction[0] == 1 else "No Flood"

    return f'''
    <h2>Prediction Result</h2>
    <h3>{result}</h3>
    <a href="/">Predict Again</a>
```

```
@app.route("/")
def home():
    return '''
    <h2>Rising Waters: Flood Prediction</h2>
    <form action="/predict" method="post">
    Rainfall:<br>
    <input name="rainfall"><br><br>
    Water Level:<br>
    <input name="water"><br><br>
    Humidity:<br>
    <input name="humidity"><br><br>
    Temperature:<br>
    <input name="temp"><br><br>
    <input type="submit" value="Predict">
    </form>
    '''
```

Activity 3.1: Writing the Main Application Logic in app.py

The main application logic of the **Rising Waters – AI-Based Flood Prediction System** is responsible for loading the historical flood dataset, preprocessing environmental data, training the K-Nearest Neighbors (KNN) machine learning model, evaluating prediction performance, and generating flood predictions. The implementation is carried out using Python in the project notebook, which serves as the core application file of the system.

The application begins by importing the required Python libraries for data analysis, visualization, and machine learning. The historical flood dataset is loaded into memory using the Pandas library. The dataset is then analyzed to identify missing values, duplicate records, and inconsistent data. Appropriate preprocessing techniques are applied to improve data quality before training the machine learning model.

The environmental parameters including rainfall, water level, humidity, and temperature are selected as input features, while the **Flood** column is considered the target variable. The dataset is divided into training and testing datasets using the `train_test_split()` function to evaluate the performance of the developed model.

The K-Nearest Neighbors (KNN) classifier is initialized and trained using the training dataset. Once training is completed, the model predicts flood conditions for unseen testing data. The predicted values are compared with the actual values to calculate the prediction accuracy.

Milestone 4: Frontend Development

Milestone 4 focuses on designing a simple and interactive user interface for the Rising Waters – AI-Based Flood Prediction System. The objective is to provide users with an intuitive platform where environmental parameters can be entered easily and flood prediction results can be viewed instantly.

Activity 4.1: Designing and Developing the User Interface

Set Up the Main Interface

The main interface is designed to provide a clean and organized layout for users to interact with the flood prediction system.

The interface contains:

- Project Title
- Project Description
- Dataset Information
- Prediction Section
- Visualization Section
- Output Section

Design the Input Section

The input section allows users to enter environmental parameters required for flood prediction.

The following input fields are provided:

- Rainfall (mm)
- Water Level (m)
- Humidity (%)
- Temperature (°C)

The entered values are validated before being passed to the trained KNN model.

Prediction Button

After entering all environmental parameters, users click the Predict Flood button.

The system performs the following operations:

- Validate the input values.
- Load the trained KNN model.
- Process the input data.
- Generate flood prediction.
- Display the result.

Output Section

The output section displays the prediction result.

Possible outputs include:

- **Flood Likely**
- **No Flood**

The prediction result is generated within a few seconds.

Activity 4.2: Creating Dynamic Content Rendering

The Rising Waters application dynamically displays prediction results and graphical visualizations after processing the environmental parameters entered by the user.

Input Processing

The application receives the following values:

- Rainfall
- Water Level
- Humidity
- Temperature

These values are converted into a feature vector and passed to the trained KNN model.

Graph Rendering

The application generates and displays:

- Scatter Plot
- Flood Risk Cluster Visualization
- Correlation Heatmap

These graphs assist users in understanding flood-related environmental patterns

Result Display

The prediction results include:

- Flood Status
- Prediction Confidence (if available)
- Dataset Visualization
- Model Accuracy

Benefits of the Frontend

- Easy to use
- Fast prediction
- Interactive visualization
- Clear result presentation
- Suitable for disaster management applications
- Can be extended into a web or mobile application

Milestone 5: Deployment and Testing

The final milestone focuses on testing, validating, documenting, and deploying the Rising Waters – AI-Based Flood Prediction System. After successfully training the K-Nearest Neighbors (KNN) model, the application is evaluated using testing data to verify its prediction accuracy and reliability. The completed project is then organized, documented, uploaded to GitHub, and demonstrated through a project presentation video.

This milestone ensures that the developed system is functional, reproducible, and ready for future enhancements and deployment.

Activity 5.1: Testing the Flood Prediction System

Testing is performed to evaluate the correctness and reliability of the developed machine learning model.

Testing Objectives

- Verify dataset preprocessing.
- Validate model predictions.
- Evaluate prediction accuracy.
- Ensure stable application performance.
- Test different environmental input values.

Testing Procedure

Step 1: Load the Testing Dataset

The testing dataset contains environmental records that were not used during model training.

Step 2: Generate Predictions

The trained KNN model predicts flood conditions using the testing dataset.

Step 3: Compare Results

The predicted values are compared with the actual flood labels.

Step 4: Calculate Accuracy

The prediction accuracy is calculated using the Accuracy Score metric.

Step 5: Validate Results

The generated prediction graphs and outputs are verified for correctness.

Performance Metrics

The developed model is evaluated using the following metrics:

- Accuracy Score
- Prediction Results
- Scatter Plot Analysis
- Cluster Visualization
- Confusion Matrix (Optional)

Expected Output

The system correctly classifies environmental conditions as:

- Flood Likely
- No Flood

The generated visualizations support the prediction results and improve interpretation.

Activity 5.2: Project Deployment

After successful testing, the project is prepared for deployment and documentation.

Project Components

The deployment package contains:

- Source Code
- Jupyter Notebook
- Dataset
- Documentation
- Output Images
- Requirements File
- README File

Software Requirements

- Python 3.10+
- Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Installation steps

```
git clone https://github.com/lingamdinne-bhanusree/Rising-Water-AI-ML
cd Rising-Water-AI-ML
pip install -r requirements.txt
jupyter notebook
```

Open **project.ipynb** and execute all cells sequentially.

Activity 5.3: GitHub Repository and Project Demonstration

Version control is maintained using GitHub, allowing efficient collaboration and project management.

GitHub Repository

Repository Name:

Rising-Water-AI-ML

Repository Link:

<https://github.com/lingamdinne-bhanusree/Rising-Water-AI-ML>

Project Demonstration

A demonstration video has been prepared to explain the complete workflow of the project.

The video demonstrates:

- Dataset Overview
- Data Cleaning

- Exploratory Data Analysis
- Model Training
- Flood Prediction
- Output Visualization
- Project Results

Data Visualization Screen

This screen presents visual representations of the dataset to help users understand the relationship between environmental parameters.

Visualizations include:

- Scatter Plot
- Correlation Heatmap
- Flood Risk Cluster Graph
- Feature Distribution Graphs

Prediction Screen

Users enter environmental values including:

- Rainfall
- Water Level
- Humidity
- Temperature

The trained KNN model processes these inputs and predicts whether flood conditions are likely.

Results Screen

After prediction, the application displays:

- Prediction Result (Flood Likely / No Flood)
- Prediction Summary
- Visual Graphs
- Dataset Analysis
- Model Performance

The results help users understand the predicted flood risk and make informed decisions.

Results

The Rising Waters – AI-Based Flood Prediction System was successfully developed and tested using historical environmental data. The K-Nearest Neighbors (KNN) machine learning algorithm was trained using features such as rainfall, water level, humidity, and temperature to predict flood occurrence. After preprocessing the dataset and training the model, the system was evaluated using a separate testing dataset. The prediction results showed that the model was able to correctly classify flood and non-flood conditions for most test samples.

The trained model effectively identified patterns in the environmental data and produced reliable predictions based on user-provided inputs.

The exploratory data analysis helped in understanding the relationships between different environmental parameters. Scatter plots and cluster visualizations revealed that higher rainfall and increased water levels were strongly associated with flood occurrence. The correlation analysis also showed that multiple environmental factors collectively influence flood risk.

The system allows users to enter environmental parameters and instantly receive a prediction indicating either **"Flood Likely"** or **"No Flood"**. This demonstrates the practical application of machine learning in supporting early flood warning systems

Conclusion

The Rising Waters – AI-Based Flood Prediction System successfully demonstrates the application of Machine Learning for flood forecasting using environmental data. By utilizing historical information such as rainfall, water level, humidity, and temperature, the K-Nearest Neighbors (KNN) algorithm effectively predicts the likelihood of flood occurrence.

The project integrates data preprocessing, exploratory data analysis, visualization, model training, and prediction into a unified workflow. The generated graphs and prediction outputs help users better understand environmental patterns and flood risks.

The developed system provides an efficient, user-friendly, and cost-effective approach to flood prediction. It has the potential to assist disaster management authorities, researchers, and local communities by providing early warnings that can reduce the impact of floods.

Demo Video Link:

<https://drive.google.com/file/d/1CaTryuRoyq9rHg4zd991KBXZvpmqNeaS/view?usp=drivesdk>

GitHub link:

<https://github.com/mythricg2005-source/Raising-Water>